Cloudflare Worker Setup Reference

Reusable guide for contact-form API deployments

Project example: acaring-contact-api

Purpose: This reference captures the clean Worker deployment pattern we validated: a dedicated Worker folder in the GitHub repo, Wrangler configuration, Cloudflare Workers Builds settings, API token permissions, secrets handling, testing, and troubleshooting. Keep this in the repository docs so future form/API Workers can be set up without repeating the same trial-and-error cycle.

Setting	Value / Guidance
Document version	1.0
Prepared date	2026-06-08
Validated use case	Cloudflare Worker API that receives website contact form submissions and sends email
Recommended Cloudflare product	Cloudflare Workers for the backend API; Cloudflare Pages remains for the static website
Primary command path	Build command: npm install; Deploy command: npm run deploy

Security reminder: never place real API keys, email provider keys, tokens, or SMTP credentials in GitHub. Use Cloudflare Worker secrets or dashboard environment variables.

Contents

1. Architecture decision

Use this split for a small static website plus a contact form backend:

Component	Cloudflare product	Purpose	Deployment model
Public website	Cloudflare Pages	Serve HTML, CSS, JavaScript, images, and static assets.	Git-connected Pages deployment.
Contact form API	Cloudflare Worker	Receive POST requests, validate payloads, send email, and return JSON.	Workers Builds connected to GitHub, rooted at contact-worker.

Why this split: The website and the API have different deployment behavior. Pages is best for static files; Workers is best for request-handling logic. Keeping the API in contact-worker prevents Pages deploy commands and Worker deploy commands from being mixed together.

2. Golden path setup checklist

- Create or keep a dedicated repo folder named contact-worker.
- Ensure contact-worker contains package.json, wrangler.toml, and src/index.js.
- Ignore .wrangler/, node_modules/, .env, and .dev.vars.
- Create a Cloudflare Worker project named acaring-contact-api.
- Connect it to the GitHub repository henyamburu/acaring-adult-home.
- Set Path / root directory to contact-worker, not /contact-worker.
- Set Build command to npm install.
- Set Deploy command to npm run deploy.
- Use a Worker deploy token, not a Pages deploy token.
- Store secret values in Cloudflare Worker secrets/environment settings, not in GitHub.
- Deploy production branch main.
- Submit a test form and confirm email delivery.
- Document the endpoint URL used by the website form.

3. Repository structure

Recommended folder layout:

```
henyamburu/acaring-adult-home/
├─ index.html
├─ contact.html
├─ styles.css
├─ script.js
├─ contact-worker/
│  ├─ src/
│  │  └─ index.js
│  ├─ package.json
│  ├─ wrangler.toml
│  ├─ .gitignore
│  └─ docs/
│     └─ contact-worker-setup.md
└─ docs/
```

```
└─ deployment-notes.md
```

Path	Required?	Purpose
contact-worker/src/index.js	Yes	Worker fetch handler. This is the runtime API code.
contact-worker/wrangler.toml	Yes	Wrangler deployment configuration. Must point main to src/index.js.
contact-worker/package.json	Yes	Defines deploy/dev scripts and pins Wrangler as a dev dependency.
contact-worker/.gitignore	Yes	Keeps local cache and secrets out of Git.
contact-worker/.wrangler/	No	Local cache/output. Do not commit.

4. Required files and templates

4.1 contact-worker/package.json

```
{
  "name": "acaring-contact-api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "wrangler dev",
    "deploy": "wrangler deploy"
  },
  "devDependencies": {
    "wrangler": "^4.98.0"
  }
}
```

4.2 contact-worker/wrangler.toml

```
name = "acaring-contact-api"
main = "src/index.js"
compatibility_date = "2026-06-08"
workers_dev = true
```

Critical field: main = "src/index.js" is what tells Wrangler where the Worker entry point is. Without this, wrangler deploy can fail with a missing entry-point error.

4.3 contact-worker/.gitignore

```
.wrangler/
node_modules/
.env
.dev.vars
```

4.4 Minimum Worker handler shape

```
export default {
  async fetch(request, env, ctx) {
    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    if (request.method !== "POST") {
      return jsonResponse({ ok: false, error: "Method not allowed" }, 405);
    }

    try {
      const payload = await request.json();

      if (!payload.name || !payload.email || !payload.message) {
        return jsonResponse({ ok: false, error: "Missing required fields" }, 400);
      }

      // Existing email provider logic goes here.
      return jsonResponse({ ok: true, message: "Contact request received." });
    } catch (error) {
      return jsonResponse({ ok: false, error: "Invalid request payload" }, 400);
    }
  }
};

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "https://acaringadulthome.com",
    "Access-Control-Allow-Methods": "POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type"
  };
}

function jsonResponse(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status,
    headers: {
      "Content-Type": "application/json",
      ...corsHeaders()
    }
  });
}
```

CORS guidance: During initial testing, some teams temporarily use Access-Control-Allow-Origin: *. For production, restrict it to the website origin, for example https://acaringadulthome.com.

5. Cloudflare dashboard values

Setting	Value / Guidance
Cloudflare area	Workers & Pages -> Create application -> Import a repository -> Create a Worker
Repository	henyamburu/acaring-adult-home
Project name	acaring-contact-api
Production branch	main
Build command	npm install
Deploy command	npm run deploy
Path / root directory	contact-worker
Non-production builds	Optional. Keep off at first unless preview Worker builds are required.
Non-production branch deploy command	If preview builds are enabled, use Cloudflare default/versioning command only after production deploy is stable.
API token	Use a Worker deploy token, not a Pages token.

Path value: Use contact-worker without a leading slash. This tells Cloudflare to run npm install and npm run deploy inside the Worker subfolder.

6. API token permissions

Create a dedicated API token for Worker deployments. Suggested token name:

```
CF Workers Deploy - acaring-contact-api
```

Permission category	Permission	Level	Why it is needed
Account	Workers Scripts	Edit / Write	Allows deployment and updates to the Worker script.
Account	Account Settings	Read	Allows Wrangler/build tooling to read account context.
User	User Details	Read	Allows token identity/account lookup during deployment.
User	Memberships	Read	Allows Wrangler to confirm account membership.
Zone	Workers Routes	Edit / Write	Needed if the Worker will be attached to a custom domain or route.

Workers Routes location: Workers Routes is normally a Zone permission, not an Account permission. If you only deploy to workers.dev and do not map a custom route yet, Workers Routes may not be required immediately, but it is useful for route-based deployments later.

Token to use	Use for	Do not use for
CF Workers Deploy - acaring-contact-api	Cloudflare Worker API deployment.	Cloudflare Pages static site deployment.
CF Pages Deploy - <project>	Pages direct upload / Pages deployment workflows.	Worker API deployment.

7. Secrets and environment variables

Data type	Where to store	Example
Public/non-sensitive config	Wrangler vars or Cloudflare dashboard environment variables	CONTACT_TO_EMAIL if not sensitive
Sensitive API keys	Cloudflare Worker secrets	RESEND_API_KEY, SENDGRID_API_KEY, MAILGUN_API_KEY
Local development values	.dev.vars or .env, ignored by Git	RESEND_API_KEY=...

```
# Add a production secret from local terminal
npx wrangler secret put RESEND_API_KEY

# Optional local-only file, not committed
contact-worker/.dev.vars
```

Security rule: Never commit .env, .dev.vars, API tokens, provider keys, SMTP passwords, or Cloudflare tokens to the repository.

8. Contact form integration contract

Item	Value / Convention
HTTP method	POST
Content-Type	application/json
Required fields	name, email, message
Optional fields	phone, preferredContactMethod, careNeed, sourcePage
Success response	{ "ok": true, "message": "Contact request received." }
Validation error response	{ "ok": false, "error": "Missing required fields" }
Production CORS origin	https://acaringadulthome.com

Example test request

```
curl -X POST "https://<worker-url>/"   -H "Content-Type: application/json"   -d '{
   "name": "Test User",
   "email": "test@example.com",
   "message": "Testing the contact form Worker."
 }'
```

9. Deployment validation checklist

- Cloudflare build completes successfully.
- Wrangler deploy log shows the Worker was uploaded/deployed.
- Worker URL responds to OPTIONS with 204 if CORS preflight is enabled.
- Worker rejects GET with 405 Method not allowed.
- Worker rejects invalid JSON with 400 Invalid request payload.
- Worker rejects missing required fields with 400 Missing required fields.
- Worker accepts a valid POST with 200 and ok: true.
- Email is received at the expected destination inbox.

- No real test data remains hardcoded in source code.
- Cloudflare secrets/environment variables are set for production.
- Website form endpoint points to the correct Worker URL or custom route.

10. Troubleshooting matrix

Symptom	Likely cause	Fix
Missing entry-point to Worker script	wrangler.toml is missing main, wrong root folder, or deploy command is running outside contact-worker.	Set Path/root directory to contact-worker and ensure wrangler.toml has main = "src/index.js".
Authentication error code 10000	Wrong API token type or missing permission.	Use Worker deploy token, not Pages token. Confirm Workers Scripts Edit/Write and needed read permissions.
npm run deploy not found	package.json missing or Cloudflare is not running inside contact-worker.	Add package.json and set Path/root directory to contact-worker.
wrangler: command not found	Wrangler not installed as dependency.	Add wrangler to devDependencies and run npm install before deploy.
Email does not send but Worker returns ok	Email provider logic not running, missing secret, or swallowed provider error.	Log provider response safely; confirm secrets are set in Cloudflare.
CORS error in browser	Worker response lacks correct Access-Control-Allow-Origin or OPTIONS handling.	Handle OPTIONS and set production origin to https://acaringadulthome.com.
Form works locally but not production	Local .dev.vars exists but production secrets are missing.	Add secrets in Cloudflare or with npx wrangler secret put.
Wrong project deploys	Cloudflare project points at repo root or the Pages project instead of contact-worker.	Check project type is Worker and Path/root directory is contact-worker.

11. Reusable Codex instruction

Use this instruction when creating or refactoring a future Worker folder:

```
Task: Set up a Cloudflare Worker API in a dedicated folder.

Folder name: <worker-folder-name>
Cloudflare Worker name: <worker-project-name>

Required structure:
<worker-folder-name>/
├─ src/
│  └─ index.js
├─ package.json
├─ wrangler.toml
├─ .gitignore
└─ docs/
   └─ worker-setup.md

Requirements:
1. Do not commit .wrangler/, node_modules/, .env, or .dev.vars.
2. package.json must include scripts:
   - dev: wrangler dev
   - deploy: wrangler deploy
3. wrangler.toml must include:
```

```
   - name = "<worker-project-name>"
   - main = "src/index.js"
   - compatibility_date = current date
   - workers_dev = true unless a custom route is required immediately
4. src/index.js must export a valid Worker default fetch handler.
5. Preserve working business logic if refactoring an existing Worker.
6. Add docs/worker-setup.md with Cloudflare dashboard values:
   - Project type: Worker
   - Root directory/path: <worker-folder-name>
   - Build command: npm install
   - Deploy command: npm run deploy
   - API token type: Worker deploy token
7. Open a PR. Do not merge automatically.
```

12. Sources and official references

Official references used when preparing this guide:

Topic	Official source
Workers Builds configuration, root directory, build/deploy commands, token behavior	https://developers.cloudflare.com/workers/ci-cd/builds/configuration/
Workers Builds advanced/monorepo setup	https://developers.cloudflare.com/workers/ci-cd/builds/advanced-setups/
Wrangler configuration file and Worker entry point	https://developers.cloudflare.com/workers/wrangler/configuration/
Wrangler CLI overview	https://developers.cloudflare.com/workers/wrangler/
Cloudflare API token templates and permission categories	https://developers.cloudflare.com/fundamentals/api/reference/template/
Worker environment variables	https://developers.cloudflare.com/workers/configuration/environment-variables/
Worker secrets	https://developers.cloudflare.com/workers/configuration/secrets/
Worker routes and route mapping	https://developers.cloudflare.com/workers/configuration/routing/routes/
Worker best practice for secrets	https://developers.cloudflare.com/workers/best-practices/workers-best-practices/